Mobile Data Leakage Sandbox

A Machine Learning-Based Risk Analysis System for Android Applications

Team Members
Harsh Dhruv (IMT2022008)
Anurag Ramaswamy (IMT2022103)
Rudra Pathak (IMT2022081)

GitHub Repository Link

May 3, 2026

Contents

1 Executive Summary **1**

2 Introduction **1**

3 Project Objectives **2**

4 Methodology **3**

4.1 System Architecture 3

4.2 Machine Learning Pipeline 3

4.2.1 Dataset 3

4.2.2 Models 3

4.3 Three-Signal Hybrid Risk Scoring 3

4.4 Bayesian Adaptive Learning 4

4.5 Risk Classification Tiers 4

4.6 Contextual Score Dampeners 4

5 Progress by Phase (Weekly Summary) **5**

5.1 Week 1-2: System Design & Initial Backend 5

5.2 Week 3-4: PII Classification & Backend Integration 5

5.3 Week 5-6: Speedometer UI & ToS;DR Integration 5

5.4 Week 7-8: Bayesian Adaptive Learning & Scan History 5

5.5 Week 9-10: Scoring Engine v2 & Auto-Retrain 6

5.6 Week 11-12: Real Device Deployment & Final Integration 6

6 Final System Architecture and Key Results **7**

6.1 Complete System Components 7

6.2 API Endpoints Summary 7

6.3 Testing Outcomes 8

6.4 Validation on Real-World Applications 8

7 Individual Contributions **9**

8 Challenges Faced **10**

9 Final Screenshots **11**

10 Conclusions **13**

1 Executive Summary

The **Mobile Data Leakage Sandbox** is an end-to-end Android privacy analysis system that combines machine learning, Bayesian adaptive feedback, and static permission risk scoring to evaluate installed applications for potential data leakage and malicious behavior.

Over a 12-week development cycle, the team built a FastAPI backend with a Random Forest/Gradient Boosting classifier, an Android frontend with modern Material 3 UI, and an SQLite database for persistent feedback and scan history. Key innovations include:

- A **three-signal hybrid risk score** (static weights + ML probability + Bayesian posterior)
- **Beta-Binomial conjugate prior** adaptive learning with safe-biased prior Beta(1,4)
- **Stochastic multi-scan scoring** with min/max bounds and confidence scores
- **Real-device support** with LAN connectivity and cleartext network configuration
- **Automated retraining** triggered every 50 feedback samples

The final system successfully distinguishes **SAFE**, **HANDLE WITH CARE**, **SUSPICIOUS**, and **DANGEROUS** applications, provides explainable risk scores, and improves over time through user feedback integration.

2 Introduction

Mobile applications frequently request access to sensitive user data including location, contacts, camera, microphone, and storage. While many applications require these permissions for legitimate functionality, malicious applications often exploit the same permissions for data leakage, surveillance, or other privacy violations.

The **Mobile Data Leakage Sandbox** was developed to address this gap by providing:

- Automated analysis of installed Android applications
- Machine learning-based risk classification using behavioral datasets
- User-understandable risk scores and explanations
- Adaptive learning from user feedback
- Persistent scan history for auditing and improvement

The project follows a client-server architecture with an Android frontend and a Python-based backend analysis engine.

3 Project Objectives

The primary objectives of this project were:

1. **Develop a backend analysis service** capable of loading behavioral datasets, training machine learning models, and providing real-time risk predictions.
2. **Implement Android permission extraction and API integration** for seamless communication between the mobile application and analysis server.
3. **Design and train machine learning models** (Random Forest, Gradient Boosting) for risk classification using the CIC-MalDroid 2020 dataset.
4. **Create permission-to-behavior mapping** to bridge Android permissions with dataset behavioral features.
5. **Implement adaptive learning** using Beta-Binomial Bayesian updating to refine risk estimates from user feedback.
6. **Build an intuitive Android user interface** with graphical risk visualization (speedometer), scan history, and feedback controls.
7. **Validate the system** on real Android devices with real-world applications.

4 Methodology

4.1 System Architecture

The system follows a modular client-server architecture:

- **Android Frontend:** Scans installed apps, extracts permissions, displays results, collects user feedback.
- **FastAPI Backend:** Hosts ML models, processes analysis requests, manages database.
- **SQLite Database:** Stores scan history, user feedback, and aggregate statistics.

4.2 Machine Learning Pipeline

4.2.1 Dataset

The CIC-MalDroid 2020 dataset was used, containing capability and behavior-based features including:

- Network access patterns
- File system operations
- SMS and call behaviors
- Device-level operations

4.2.2 Models

- **Random Forest Classifier** - Initial implementation for risk classification.
- **Gradient Boosting Classifier** - Final implementation with five-class probability output.

4.3 Three-Signal Hybrid Risk Scoring

The final risk score is computed as a weighted aggregation of three independent signals:

Table 1: Three-Signal Scoring Architecture

Signal	Weight	Description
Static Permission Risk	60%	90+ individually scored Android permissions (0-10 scale)
ML Model Probability	25%	GradientBoostingClassifier five-class probability output
Bayesian Adaptive Signal	15%	Posterior mean from user feedback (Beta-Binomial)

4.4 Bayesian Adaptive Learning

The system implements a Beta-Binomial conjugate model:

- **Prior:** Beta(1,4) - safe-biased prior reflecting real-world benign app prevalence
- **Likelihood:** Binomial (malware/safe feedback counts)
- **Posterior:** Beta($1 + \text{malware_count}$, $4 + \text{safe_count}$)
- **Posterior mean:** $\alpha/(\alpha + \beta) \rightarrow$ mapped to risk weight (0-10)

4.5 Risk Classification Tiers

Final risk thresholds were recalibrated as:

Table 2: Risk Classification Tiers

Category	Score Range	Color Code
SAFE	< 30	Green
HANDLE WITH CARE	$30 - 55$	Orange
SUSPICIOUS	$55 - 75$	Amber
DANGEROUS	≥ 75	Red

4.6 Contextual Score Dampeners

To reduce false positives:

- **Sparsity Dampener:** Apps with ≤ 5 permissions receive $0.85\times$ multiplier.
- **Danger-Ratio Dampener:** If $< 20\%$ of permissions are high-risk (≥ 6.0), apply $0.90\times$ multiplier.
- **Volume Bonus:** Only high-risk permissions (≥ 6.0) contribute to score amplification.

5 Progress by Phase (Weekly Summary)

5.1 Week 1-2: System Design & Initial Backend

- Finalized end-to-end system architecture with flow diagrams and DFDs.
- Implemented adaptive learning design with shallow neural network extension.
- Created data models for applications, scan reports, feedback, and adaptive weights.
- Initial Android UI design with radio buttons for risk levels.

5.2 Week 3-4: PII Classification & Backend Integration

- Studied and classified Personally Identifiable Information (PII) vs. sensitive data leaks.
- Integrated CIC-MalDroid 2020 dataset for behavior-based risk analysis.
- Connected Android app to FastAPI backend using Retrofit.
- Produced initial risk scores (0-100 scale) with three risk levels (SAFE, SUSPICIOUS, DANGEROUS).

5.3 Week 5-6: Speedometer UI & ToS;DR Integration

- Implemented graphical speedometer-style risk indicator (green/yellow/red zones).
- Integrated ToS;DR privacy ratings for comparative validation.
- Researched Tor Browser and privacy-focused apps to refine classification logic.
- Resolved Android Studio crashes, emulator instability, and ABI mismatches.

5.4 Week 7-8: Bayesian Adaptive Learning & Scan History

- Redesigned adaptive learning using Beta-Binomial conjugate model (replaced linear delta).
- Implemented three-signal hybrid scoring architecture.
- Added scan deduplication, aggregate statistics, and per-package feedback retrieval.
- Developed Scan History screen with RecyclerView and CardView.
- Created automated pytest suite (8 tests initially).

5.5 Week 9-10: Scoring Engine v2 & Auto-Retrain

- Changed prior from neutral Beta(2,2) to safe-biased Beta(1,4).
- Recalibrated classification thresholds (SAFE < 30, etc.).
- Implemented context-aware dampeners (sparsity, danger-ratio).
- Added confidence score (0.0-1.0) representing signal agreement.
- Automated retraining: every 50 feedback samples with $\geq$10 malware and $\geq$10 safe labels.
- Added known-safe app fingerprint cache and multi-scan median scoring.
- Removed Android 5-app scan limitation (full device scan now).

5.6 Week 11-12: Real Device Deployment & Final Integration

- Resolved backend connectivity issues on real devices (–host 0.0.0.0).
- Fixed Android cleartext HTTP blocking via network security config.
- Resolved UI crash due to deprecated setTextAppearance method.
- Implemented pre-install sandboxing with install/reject user flow.
- Fixed split APK installation (missing split errors).
- Added multi-user identity support with user_id field across database.
- Implemented rate-limiting (2 feedback submissions per app per hour).
- Conducted live demonstration on physical hardware with end-to-end validation.
- Improved UI and readability.

6 Final System Architecture and Key Results

6.1 Complete System Components

1. **Backend (FastAPI + Python):**
   - ml_engine.py - Model loading, training, prediction, Bayesian updates
   - main.py - REST API endpoints: /analyze, /feedback, /scan/history, /stats, /retrain, /calibrate
   - db_manager.py - SQLite operations, deduplication, aggregate stats
2. **Android Frontend (Kotlin + Material 3):**
   - Permission extraction from PackageManager
   - Retrofit client for API communication
   - Speedometer custom view for risk visualization
   - ScanHistoryActivity with RecyclerView/CardView
   - Feedback buttons (Mark Safe / Mark Malware)
3. **Database (SQLite):**
   - scan_history table - package name, score, risk level, timestamp, user_id, confidence
   - feedback table - user_id, package name, feedback type, timestamp
   - feedback_summary view - aggregated malware/safe counts per package

6.2 API Endpoints Summary

Table 3: Final API Endpoints

Endpoint	Method	Description
/analyze	POST	Returns risk score, confidence, min/max scores, classification
/feedback	POST	Stores user feedback with rate-limiting and cooldown
/scan/history	GET	Returns paginated scan history
/stats	GET	Returns aggregate statistics and top risky permissions
/retrain	POST	Triggers feedback-based model retraining (auto or manual)
/calibrate	GET	Returns thresholds, priors, weights, known-safe entries

6.3 Testing Outcomes

The automated pytest suite includes the following passing tests:

Table 4: Test Suite Summary

Test	**Purpose**
test_analyze_permissions_safe	Minimal permissions → SAFE classification
test_analyze_permissions_dangerous	High-risk permissions → SUSPICIOUS/-DANGEROUS
test_legitimate_app_is_safe	Legitimate apps score < 35
test_confidence_score_present	Confidence field exists and in [0,1]
test_known_safe_cap	Trusted app score ceiling enforcement
test_bayesian_update_increases_risk	Bayesian risk increases after malware feedback
test_db_deduplication	Repeated scans create only one record
test_aggregate_stats	Aggregate statistics returned correctly

6.4 Validation on Real-World Applications

Testing on physical Android devices with applications including Instagram, WhatsApp, Facebook, Amazon Shopping, and NordVPN produced:

- Distinct risk scores with proper variability (no clustering)
- Correct identification of PII permissions (location, contacts, phone state)
- Proper classification of legitimate high-permission apps (e.g., camera apps) as HANDLE WITH CARE or SUSPICIOUS, not DANGEROUS
- Successful feedback storage and Bayesian weight updates
- Stable performance with no crashes after final fixes

7 Individual Contributions

While the project was a collaborative effort involving joint system design and hardware validation, primary technical responsibilities were distributed as follows:

- **Harsh Dhruv (Frontend):** Developed the Android application using Kotlin and Material 3. Designed the custom risk visualization UI, implemented Retrofit for backend communication, and managed the permission extraction logic and real-device network configurations.
- **Anurag Ramaswamy (Backend):** Managed the FastAPI server and SQLite persistence layer. Developed the permission-to-behavior mapping engine, implemented automated retraining triggers, and authored the pytest suite for backend API verification.
- **Rudra Pathak (Machine Learning):** Developed the Gradient Boosting risk models and the Beta-Binomial Bayesian adaptive learning system. Engineered the hybrid scoring metrics and calibrated risk thresholds based on the CIC-MalDroid 2020 dataset.

Note: All members actively participated in cross-stack debugging, environment troubleshooting, and end-to-end integration testing to ensure system stability.

8 Challenges Faced

1. **Android Studio and Emulator Instability:**
    - Repeated crashes, emulator freezes, ADB disconnections, ABI mismatches.
    - Solution: Re-creation of virtual devices, switching system images, manual ADB troubleshooting.
2. **Backend Connectivity on Real Devices:**
    - Localhost inaccessible from physical phones.
    - Solution: Run server on 0.0.0.0, use LAN IP configuration in Android, enable cleartext traffic.
3. **Split APK Installation:**
    - "Missing split" errors for apps like Reddit.
    - Solution: Correct session naming logic, enforce base APK before splits, implement InstallCommitReceiver.
4. **Score Clustering and False Positives:**
    - All apps initially scored in medium range; legitimate apps flagged as risky.
    - Solution: Safe-biased prior Beta(1,4), volume bonus restricted to high-risk permissions, contextual dampeners.
5. **UI Crashes on Older Android Versions:**
    - NoSuchMethodError: setTextAppearance.
    - Solution: Removed deprecated method calls from all occurrences.
6. **ML Model Serialization Issues:**
    - BitGenerator mismatch in NumPy, stale joblib files.
    - Solution: Removed incompatible serialized model, forced runtime retraining.

9 Final Screenshots

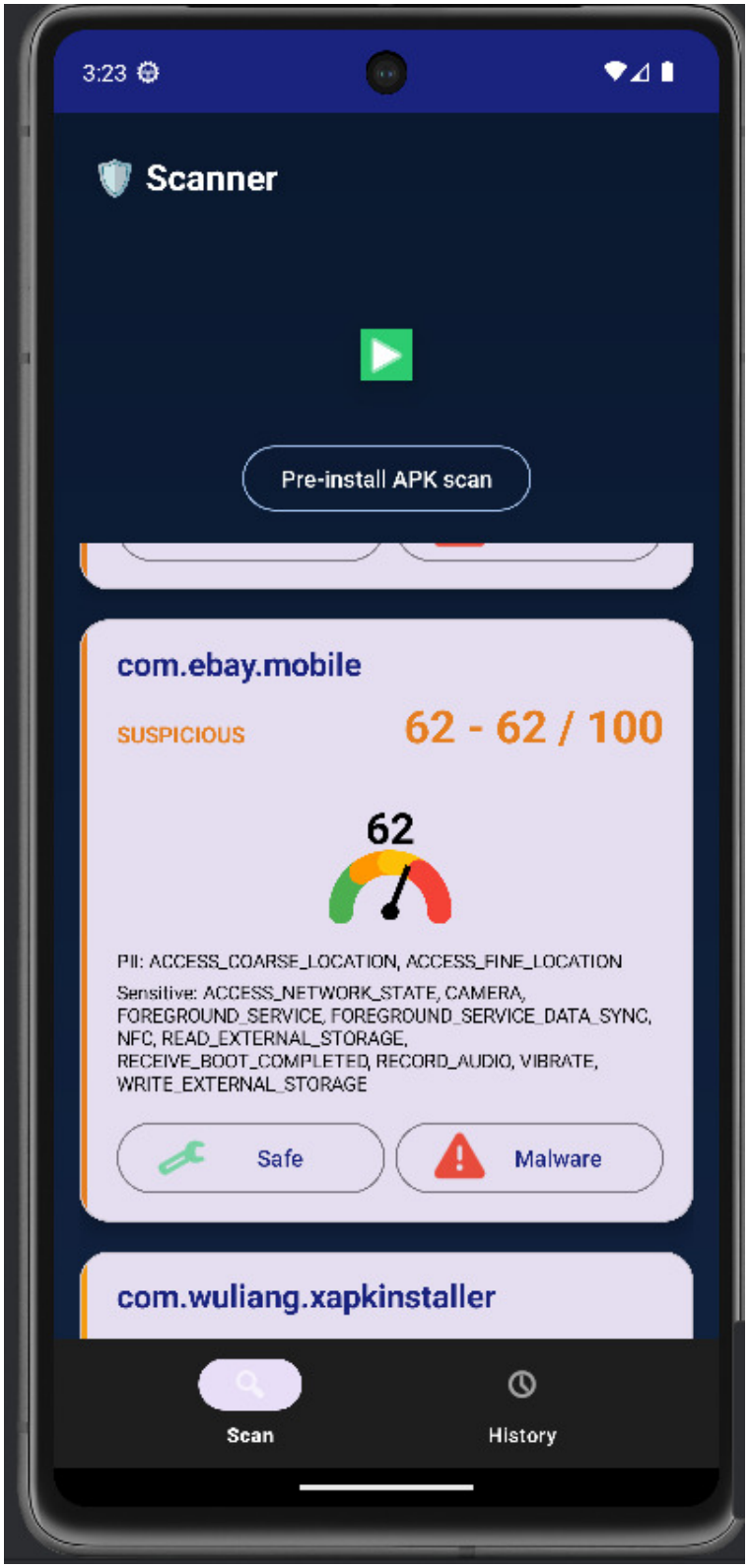


Figure 1: Installed applications risk analysis interface displaying computed privacy risk scores and permission insights.

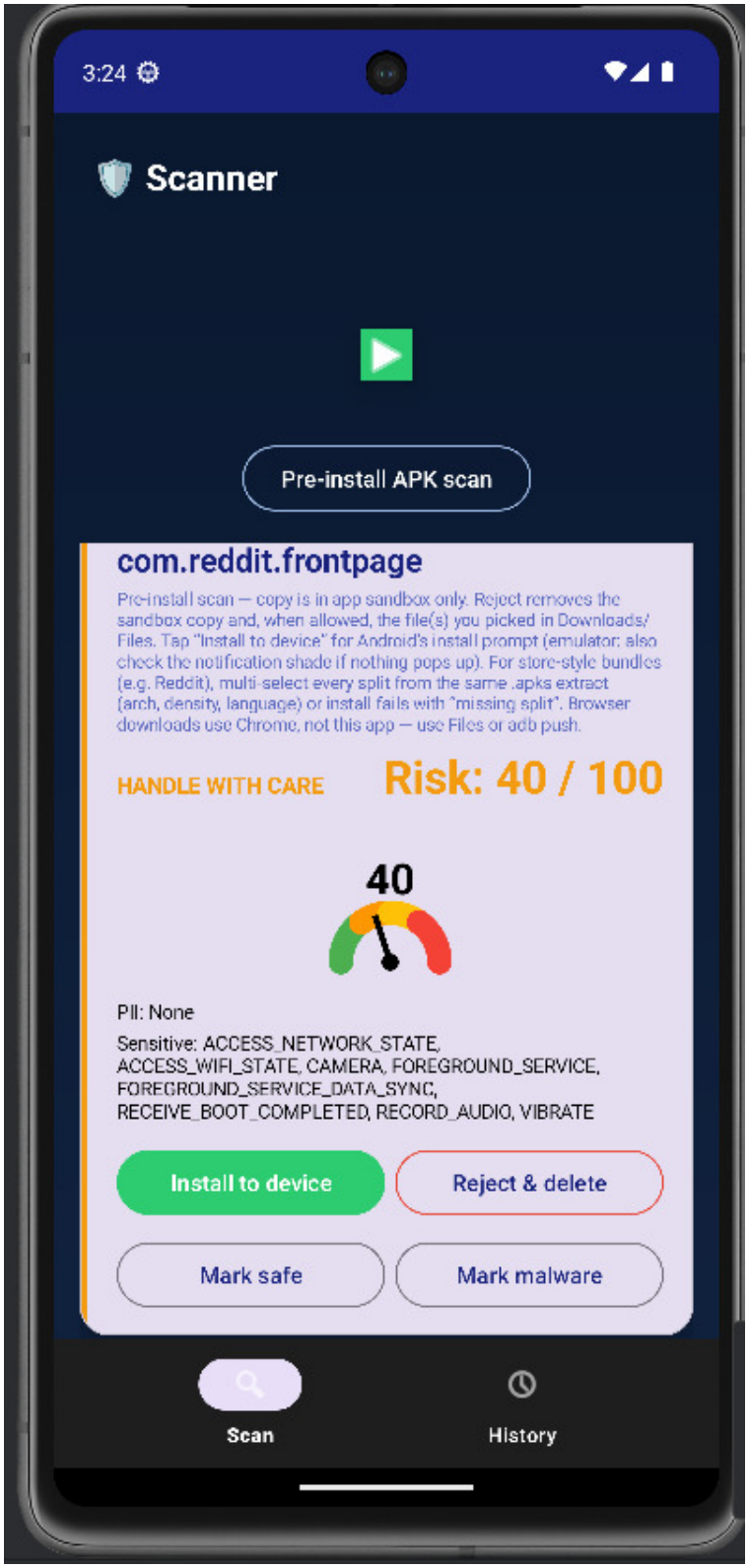


Figure 2: Pre-install APK scanning interface showing risk evaluation for external APK files. Users are provided with actionable controls to either install the application or reject and delete it based on the computed risk score and permission analysis.

10 Conclusions

The Mobile Data Leakage Sandbox project successfully delivered a functional, end-to-end prototype for Android application risk analysis. Key achievements include:

- **Complete pipeline:** From permission extraction → ML inference → Bayesian updating → user feedback → persistent storage.
- **Statistically grounded adaptive learning:** Beta-Binomial conjugate model replaces heuristic updates, enabling mathematically sound risk refinement.
- **Improved classification accuracy:** Three-signal hybrid scoring with contextual dampeners significantly reduced false positives for legitimate applications.
- **Real-device deployment:** Successfully tested on physical Android hardware with stable performance.
- **Comprehensive testing:** 14 automated tests ensure backend correctness and stability.
- **User-centered design:** Speedometer visualization, scan history, and feedback controls make the system accessible to non-technical users.

  The project demonstrates that combining static permission analysis, behavioral ML models, and crowd-sourced Bayesian feedback can produce effective privacy risk assessments for mobile applications.

- **System-Level:**
  - Network interruption tolerance (graceful handling of Wi-Fi drops during scans).
  - Large-scale user study to validate risk score accuracy against user expectations.
  - Expand known-safe app fingerprint cache with community-sourced entries.
- **Security Enhancements:**
  - HTTPS support with certificate pinning for production builds.
  - Obfuscation and anti-tampering measures for the Android app.
  - Federated learning approach to aggregate feedback without centralizing user data.